

이윤선

AI Inference/Serving Engineer

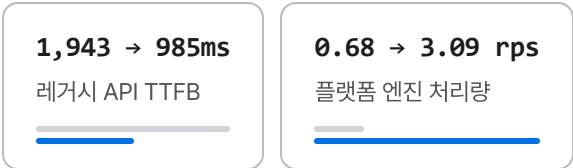
6년차 · 모델 추론 서빙과 인프라 — 음성 · LLM · 비전 · 검색

모델을 서비스로 만드는 구간에서 병목을 계측으로 찾아 지연과 처리량을 바꾸는 일을 합니다. TTS 스트리밍의 첫 응답을 절반으로 줄이고, 신규 서빙 플랫폼의 엔진 처리량을 4.5배로 끌어올렸습니다. 20여 개 모델이 도는 Kubernetes 클러스터의 관측 스택 5종도 직접 구축해, 병목을 눈으로 볼 수 있게 만들었습니다. 음성·LLM·비전 도메인을 거치며 모델 개발부터 서빙·인프라까지 다뤘고, 논문 7편과 특허 2건을 썼습니다.

핵심 성과

실시간 TTS 서빙

구간 계측으로 병목을 엔진 디코드 루프로 좁힌 뒤, 문장 단위 스트리밍으로 첫 응답을 줄이고 FP8 양자화-executor 백엔드 교체로 처리량을 올림.



- FastAPI
- WebSocket
- vLLM

MLOps 인프라

3인 팀의 사내 첫 Kubernetes 구축에서 관측성 전 계층을 맡음.

Prometheus·Grafana·AlertManager로 지표와 경보를, 온프레미스라 Elasticsearch·Loki를 직접 설치해 로그를 세움.

5종

직접 구축한 관측 스택

20개 이상

관측 대상 ML 모델

Kubernetes

Docker

CI/CD

연구 성과

센서 퓨전 기반 3D 시맨틱 세그멘테이션 연구.

자율주행·국방 도메인에서 논문·특허로 성과 입증.

7편

논문

2건

특허 (제1발명자)

우수 논문상

한국국방기술학회

PyTorch

Research

Academic

기술 스택

다른 도메인

음성 (TTS · STT · 화자 분리)

LLM · RAG · Agent

멀티모달 검색

비전 (세그멘테이션 · 탐지 · 생성)

시계열 · 이상탐지

LANGUAGES

Python

TypeScript

SQL

AI / ML

vLLM

PyTorch

Whisper

ONNX Runtime

LangGraph

SERVING · BACKEND

FastAPI

WebSocket

gRPC

Redis

DEVOPS · INFRA

Kubernetes

Docker

Jenkins · ArgoCD

근무 경력

- **MiCo AI** (구 에이아이세스) · AI Engineer · Serving
2025.04 ~ 현재 · 음성 AI 서빙
- **인피닉** · AI / MLOps Engineer
2022.08 ~ 2025.04 · 인프라와 비전 · 생성모델 연구
- **이든티앤에스** · AI Engineer
2022.04 ~ 2022.08 · OCR 데이터 도구
- **큐헛지** · Intern
2020.09 ~ 2021.09 · 금융 데이터 파이프라인
- **KISTI** · Part-time
2017.10 ~ 2018.01

연락처

이메일

yoons7829@gmail.com

LinkedIn

[linkedin.com/in/yoonseonlee](https://www.linkedin.com/in/yoonseonlee)

블로그

taepseon.tistory.com

휴대폰

010-5780-2898

GitHub

github.com/ckc5800

Notion

[포트폴리오 문서](#)

프로젝트

실무 경력

추론 서빙 · 성능 엔지니어링 4

Qwen3-TTS 플랫폼 구축

MiCo AI

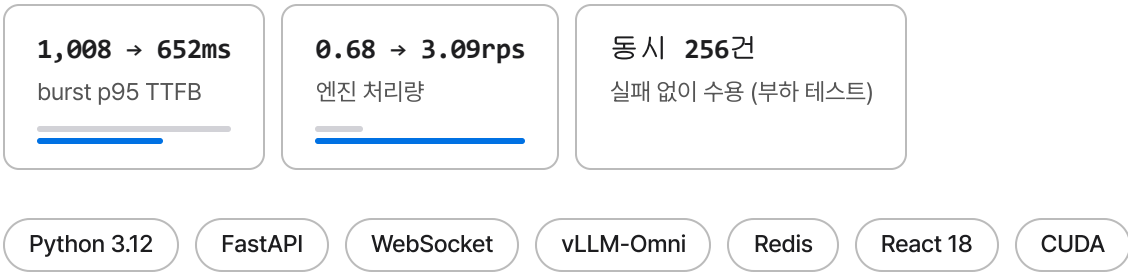
2026.03 ~ 현재 · PM · 풀스택 단독 개발 (백엔드 · 프론트 · 인프라)

문제 상담센터에 제안할 자사 TTS 솔루션이 없었음. 실시간 상담에 쓰려면 응답 속도와 음성 선택지를 갖춘 제품을 새로 만들어야 했음

해결 Qwen3-TTS 기반으로 신규 구축 — vLLM(GPU)·Supertonic(CPU) 듀얼 엔진의 추론 서빙 아키텍처를 단독 설계.

지연은 구간 계측으로 분해해 병목을 엔진 디코드 루프로 좁힌 뒤 문장 단위 스트리밍으로 첫 응답을 당기고, 처리량은 FP8 양자화 ·executor 백엔드 교체·vLLM 0.24 마이그레이션으로 4.5배까지 올림.

고객사 PoC·시연까지 진행



배경

상담센터를 타깃으로 제안할 자사 TTS 솔루션을 신규 구축하는 과제.

기존 시스템 교체가 아니라, 실시간 상담에 쓸 수 있는 응답 속도와 음성 선택지를 갖춘 제품을 처음부터 만드는 것이 목표였음.

모델 선정부터 서빙 아키텍처, 백엔드·프론트엔드, 배포·운영까지 전 구간을 단독으로 맡았고, 고객사 PoC·시연까지 진행.

차별점

① 스트리밍 품질 자동 감지 · 자가 치유 — 잘리거나 반복된 음성을 서버가 스스로 걸러내는 구조.

- 합성 오디오의 **예상 길이 대비 실제 길이 비율**로 잘림·반복을 판정 (0.35배 미만 = 잘림 의심, 3배 초과 = 반복 의심).
텍스트에서 음절 버킷으로 기대 길이를 추정
- 이상 클립은 캐시에 넣지 않고 배제.
같은 문장의 다음 요청이 정상 합성을 받아 **자가 치유**.
스트리밍은 이미 나간 청크를 되돌릴 수 없으므로 후속 요청부터 회복되는 구조
- 감지(로깅)는 전 요청·전 언어 상시 동작.
캐시 배제는 오탐이 정상 클립을 버리는 쪽이 더 위험하다고 판단해, 길이 추정이 신뢰되는 언어(CJK)로 게이팅하고 플래그로 단계 도입 — 현재 운영 관측 단계

㉔ 처리량 천장 진단 → 돌파 — "1.7 rps가 천장"이라던 사내 결론을 벤치마크로 반증.

- 기존 사내 결론은 **"약 1.7 rps는 버전 무관한 구조적 천장"**.
버전(0.22/0.24) × executor(mp/uni) **2×2 전수 벤치**로 이를 반증
- 처리량의 몸통은 보코더(Code2Wav)가 launch-bound eager bs≈1로 돌던 것이 **CUDA-graph replay + 요청 간 배치 디코딩**으로 바뀐 것.
배치 기계장치는 구버전 이미지에도 있었으나 자체 운영 config(eager 강제·EOS 튜닝·토큰예산)가 봉인하고 있었고, 네이티브 EOS·abort 레이스 수정이 그 봉인을 풀어줌
- 동일 부하(conc8)에서 **0.73 → 1.61 rps**, 각자 최대 안정치 **0.73 → 3.09 rps(4.2배)**.
자체 패치 3종·EOS 튜닝·문장분할 의존도 함께 제거
- 안정성도 질적 전환.
conc16 진입 시 엔진 사망(3회 재현)하던 것이 conc48 무오류(1,500+ 요청)로

㉕ 용량 한계 실측 — 동시 몇 명까지 받을 수 있는지를 숫자로 확정.

- burst 스왑으로 SLA 확립.
TTFB ≤**500ms 동시 15** · ≤**1000ms 동시 30**, 수용 한계 36 (실링 약 40)
- 판정 기준은 지연이 아닌 **RTF 여유**.
코덱 좌측 컨텍스트를 조정해 N30 RTF 0.91 → 0.71 확보, N40은 0.96~1.04로 마진이 사라지는 것을 확인하고 운영값을 실링 -1 티어(36)로 결정
- 자원 측도 함께 실측.
per-stage VRAM(stage0 18.9G + stage1 7.2G) · 컨테이너별 CPU/메모리 피크 · GPU 온도와 SM 클럭까지 기록해 열 스로틀링 드리프트를 측정 오염과 구분

성과

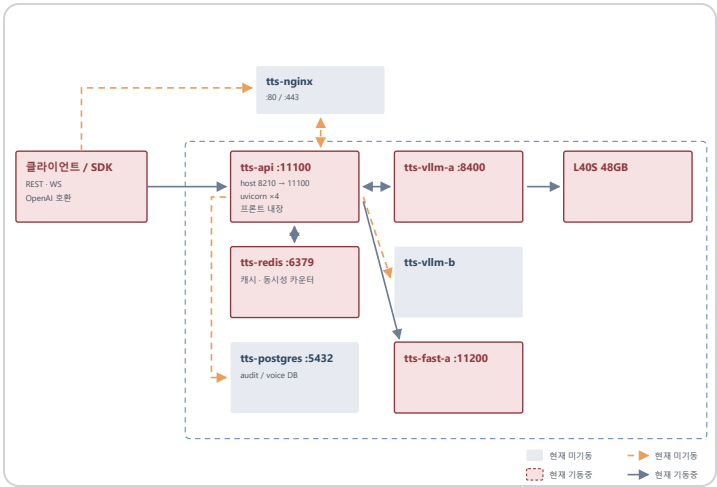
- 자체 튜닝(FP8-executor)으로 **0.68 → 1.41 rps**, 엔진을 옮긴 뒤 최대 안정치 **3.09 rps**.
측정 조건별 비교는 차별점 ②에 정리
- 부하 테스트를 3층으로 설계 — 스모크(생사 확인) → 순간 폭주(동시 4~256 워커가 같은 순간 도착) → 지속 부하(Poisson 도착 모사, 텍스트 길이 3종·화자 5명·캐시 히트 30% 반영, 시나리오별 3분).
Stream 기준 **동시 256건까지 실패 없이 수용**, 정상 부하 **TTFB p50 70ms**(p95 219 · p99 292ms), 최대 처리량 **8.7 rps** (2026.07 실측)
- 32개 언어 지원 단일 플랫폼 (vLLM 10개 + Supertonic 22개)
- ITN 정규화 정확도 50.1% → 70.9% (검증셋 기준)
- 같은 GPU에서 **burst p95 TTFB 1,008ms → 652ms** (동시 20 기준). ≤500ms는 달성 불가에서 동시 15로, ≤1000ms는 20~25에서 30으로 올랐다.
동시 스트림 수용 한계 36 실측 (2026.07 말)
- TTFB 경로의 순차 Redis 왕복을 16회에서 13회로 줄이고, 문장 캐시는 일괄 조회로 바꿔 N왕복을 1왕복으로

오픈소스 기여 (vLLM-Omni)

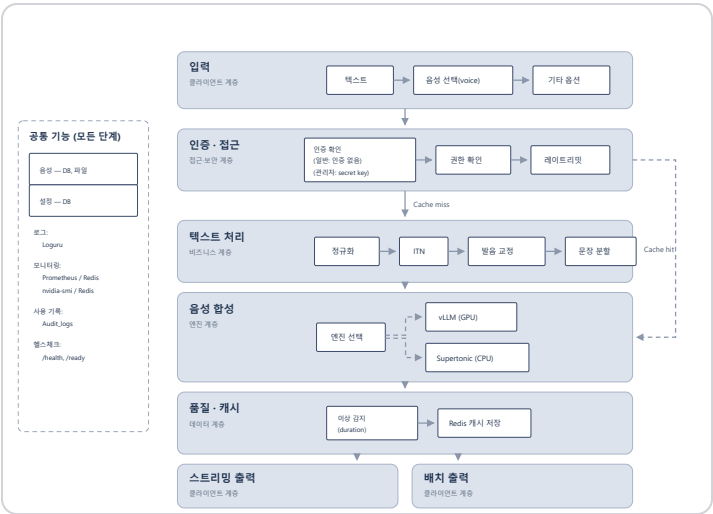
- 운영 중 stage0(talker) 프로세스의 호스트 메모리가 선형 증가하고 끝내 플래토에 도달하지 않는 것을 발견.
추론 엔진이 **abort된 요청**의 sender 측 상태(코덱 프레임 이력·참조 텐서·체크 카운터)를 회수하지 않는 것이 원인 — 회수 경로가 "terminal 체크 전송 성공"에만 걸려 있어, 클라이언트가 끊고 나간 요청은 프로세스가 죽을 때까지 남는 구조
- **인과를 수치로 좁힌 방법** — 부하-메모리 할당자·캐시 설정을 바꾼 4개 구성에서 증가율은 22~296 MiB/h로 **13배** 차이가 나는데, abort 건수로 나누면 **36~56 KB/abort로 일정했다**.
"부하가 높아서"가 아니라 "abort 하나당" 새는 것임을 이 불변량으로 확정하고, memray 스냅샷(10분 vs 20분 창)으로 해당 호출 경로의 미해제 할당이 창 길이에 비례해 늘어나는 것을 교차 확인
- abort 시점에만 sender 상태를 회수하는 최소 패치로 A/B 검증 — **257 → 60 MiB/h (77% 감소)**, 4.4시간 창 $R^2 \approx 0.99$.
같은 런의 미패치 대조군은 15.2시간 동안 258 MiB/h, $R^2 = 1.00$
- 고치고 끝내지 않고 **잔여 ~13 KB/abort**가 남는 것까지 측정해, 원인을 save 큐에 이미 적재된 체크가 상태를 되살리는 레이스로 특정.
상위 RFC가 이미 구현해 둔 지연 정리 패턴(`_deferred_send_cleanup`)을 이식하면 닫힌다는 것까지 이슈에 남겼다
- 이슈로 리포트한 뒤 같은 원인을 다루는 기존 PR을 발견하고, 그쪽에 프로덕션 검증 데이터를 붙였다.
PR 작성자와 리뷰어가 근거로 인용 (2026.08, 머지 대기)

이슈 #6352 (리포트) (<https://github.com/vllm-project/vllm-omni/issues/6352>) · **PR #4349 (검증 데이터 제공)** (<https://github.com/vllm-project/vllm-omni/pull/4349>)

시스템 구조



물리 구조 — 컨테이너 단위 배치.
실선이 가동 중인 경로, 점선은 미가동(nginx·PostgreSQL·vllm-b).



논리 구조 — 요청이 거치는 6개 계층.
캐시 히트면 텍스트 처리와 합성을 건너뛴다.

구현

시스템 아키텍처: FastAPI WebSocket/REST 게이트웨이 + vLLM-Omni(Qwen3-TTS) 추론 엔진 + Redis 분산락 & 캐싱 + React 대시보드.

문장 단위 논리 청킹 및 WebSocket 스트리밍으로 TTFB 단축.

성능 최적화: L40S 단일 GPU 환경에서 처리량과 지연을 함께 끌어올림.

- FP8 양자화 · executor backend 최적화 · 동시성 튜닝으로 추론 처리량 개선
- Pre-Normalization · CUDA Graph Warmup · 세마포어 동시성 제어로 GPU 메모리 활용 개선
- Redis 캐싱으로 동일 문장 재요청 시 즉시 스트리밍 개시 (캐시 히트 TTFB 50ms 미만)

지연 구조 분해: 튜닝에 앞서 TTFB가 어느 구간에서 나오는지부터 체크.

엔진 패치에 전처리(prepare)와 엔진 첫 청크(큐+prefill+보코더) 지표를 심어 분해.

- 전처리 3~6ms·API 약 100ms는 고정이고, **동시 요청 수에 선형 비례하는 구간은 엔진 디코드 루프**(동시 +1당 약 +20ms)임을 확인
- 스트림 세마포어를 32→40으로 여는 통상적 처방은 대조 실험 결과 대기 위치만 엔진 안으로 옮길 뿐 SLA는 오히려 악화.
세마포어의 역할을 병목 제거 장치에서 과부하 보호막으로 다시 규정
- 실패 레버를 코덱 좌측 컨텍스트(72→25, CUDA graph capture 목록 정합 수정 동반)와 첫 청크 프레임 수(4→2) 둘로 좁혀 확정
- 청크 경계 불연속을 z-검정으로 자동 판정하는 품질 게이트를 통과시킨 뒤 채택

측정 하네스: 성능 스윙을 도구화하던 중 "0.6B 라벨로 1.7B를 측정"한 오라벨 사고를 겪고, 측정 자체를 믿을 수 있게 만드는 안전장치를 하네스에 내장.

- 서빙 중인 모델 3중 검증 — 환경변수·엔진 응답·부하기 재검증.
불일치 시 스왑 시작 불가
- 파라미터 반영 확인 실패 시 해당 값 스kip, SLA 판정에 연속성 요구(스파이크 하나로 뚫린 상위 동시성 채택 금지)
- 측정 오염 제거 — 1초마다 폴링하던 nvidia-smi가 GPU를 멈춰 세우던 것을 찾아 주기 조정, 동일 문장 반복의 prefix cache 낙관은 가변 문장 모드 병행 측정으로 해소

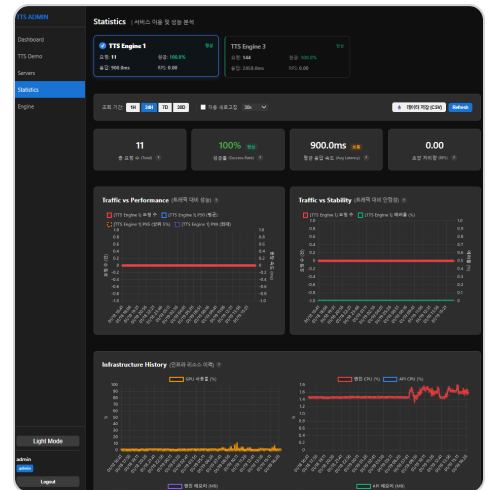
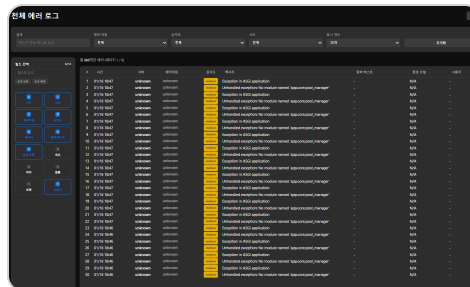
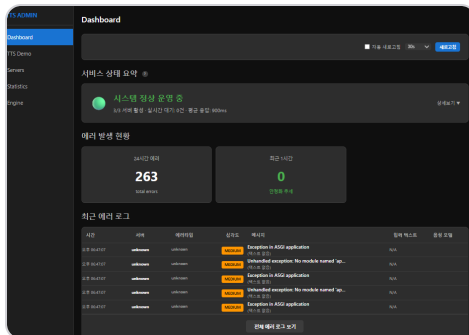
기능 확장: 참조 음성 5초만으로 화자 목소리를 복제하는 ICL 기반 음성복제 기능 구현.

- OpenAI TTS API 호환 인터페이스 + 기존 WebSocket 프로토콜 호환 레이어 — 외부 서비스가 코드 수정 없이 연동
- Redis 기반 Atomic quota·분산락·Circuit Breaker 등 장애 대응 로직
- Whisper 기반 CER/WER 자동 품질 측정 파이프라인

운영: vLLM(GPU)과 Supertonic(CPU) 듀얼 엔진 구성 — 장애 시 Circuit Breaker가 CPU 엔진으로 자동 폴백.

- 무중단 롤링 재배포 체계 + 온보딩 가이드·운영 런북·장애복구 절차·성능 명세 문서화로 운영 이관 가능한 수준까지 정비
- 배포 직후 활성 음성별 prefill 워밍과 정형 인사말 프리캐시를 자동 실행 — 첫 사용자가 콜드 경로를 밟지 않게
- 스트리밍 품질을 자동 감지해 자가 치유 (위 차별점 ① 참조).
품질 사고가 캐시에 굳는 것을 차단

운영 대시보드



데모 영상

TTS Full Synthesis

(https://drive.google.com/file/d/18mJfKttw0on_zgHEbCDJG9VIUw9YXUFe/view?usp=drive_link)

배포 및 운영

(https://drive.google.com/file/d/1FE4p0ItxPPuKvGDC90gHxbmmGTHef8YB/view?usp=drive_link)

핵심 엔지니어링

1. 스트리밍 팝 노이즈 — 스트리밍 중 간헐적으로 '틱' 잡음 발생.

PCM 청크가 홀수 바이트로 끊겨 와 16bit 샘플 정렬이 깨지는 것이 원인.

잔여 바이트를 다음 청크에 이어 붙이는 Residual Buffer로 정렬을 보장하고, 청크 경계에 Micro-fade를 적용해 해결

2. 스토리지 누수 — 보이스를 삭제해도 디스크 사용량이 계속 증가.

원본을 지워도 파생된 MD5 해시 캐시 파일이 남는 구조여서, 파생 파일까지 추적해 함께 지우는 Deep GC를 구현해 차단

3. 웹소켓 세션 탈취 방어 — 비밀번호를 바꿔도 이미 발급된 토큰으로 웹소켓 연결이 열리는 허점 발견.

토큰 발급 시점과 password_changed_at을 대조해 변경 이전 토큰을 전부 무효화

4. ITN 정규화 — URL·이메일을 한국어로 부자연스럽게 읽는 문제.

자연스러운 발음으로 변환하는 스마트 ITN 엔진을 직접 개발해 정규화 정확도 50.1% → 70.9%

5. CPU 스파이크 (ReDoS 가드 오버헤드) — 정규식 치환의 ReDoS 방어용 ThreadPoolExecutor(max_workers=1) 가드가

치환마다 새로 인스턴스화되어 다수 합성 시 CPU 스파이크와 지연을 유발.

오버엔지니어링된 가드를 제거하고 네이티브 C 구현체(re.sub) 동기 호출로 교체해 핫 패스 병목 제거

6. 전면 백색소음 장애 — vLLM 0.24 전환 직후 모든 합성이 백색소음으로 재생되는 문제를 추적해, 엔진의 SSE(JSON) 응답이 그대로

PCM으로 재생된 것이 원인임을 확인.

응답 포맷을 오디오로 강제해 해결

7. 스트리밍 WAV 헤더 정리 — 엔진이 문장마다 RIFF 헤더를 붙여 보내 클라이언트 디코딩이 어긋나는 문제.

문장별 헤더를 건너내고 세션 전체에 스트리밍 WAV 헤더를 한 번만 내보내는 구조로 정리해 해결

8. 크래시로 오진할 뻔한 스트림 집단 절단 — 고부하 burst 중 스트림이 무더기로 끊기고 수 분간 무응답.

엔진 크래시로 보였으나 컨테이너 재시작 횟수가 0인 것을 확인하고 가설 폐기.

실체는 추론 엔진의 텐서 결합 오류가 집단 발생하자 API 서킷 브레이커가 열려 빈 응답을 반환한 연쇄.

동시 한도를 실링(40)보다 한 티어 낮춰(36) 과부하 진입 자체를 완화

9. HTTP 스트림 tail 지연 — 저지연을 표방하면서 실제로는 tail 버퍼를 문장 끝까지 붙들어, 약 615ms 분량이 모일 때까지 한 바이트도

내보내지 않던 구조.

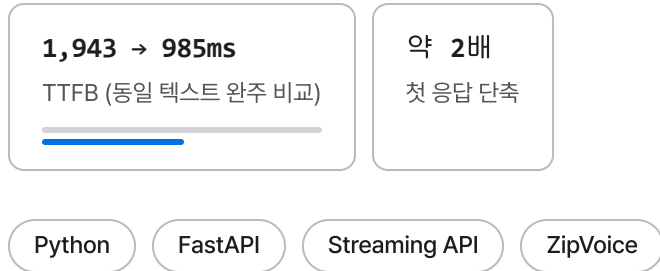
WebSocket 경로에만 있던 무음 감지 기반 조기 flush를 HTTP에 이식해 단문 TTFB와 문장 경계 끊김을 함께 해소

TTS 스트리밍 API 리팩토링 (ZipVoice 기반) MiCo AI

2025.09 ~ 2025.10 · 스트리밍 API 담당 (인수인계, 기여도 100%)

문제 전체 텍스트를 다 합성한 뒤 전송하는 구조라, 문장이 길수록 첫 응답이 비례해 늦어짐

해결 엔진의 gRPC 서버 스트리밍을 써서 문장 단위 합성 → 즉시 전송하는 엔드포인트 신설



배경

사내에서 쓰던 기존 TTS 엔진(ZipVoice)의 스트리밍 API가 담당 대상.

전체 합성이 끝나야 첫 청크가 나가는 구조라 텍스트가 길수록 첫 응답이 비례해 늦어졌고, 실시간 상담에 쓰기 어려웠음.

기존 담당자에게 인계받아 리팩토링을 맡음.

구현

SSE 청크 디코딩 실패 해결: 브라우저 `decodeAudioData()` 가 두 번째 청크부터 실패한다는 사내 사용 부서(API 호출측)의 제보를 추적.

- 원인 — 엔진이 첫 청크에만 RIFF WAV 헤더를 붙이고 이후는 raw PCM만 반환
- 조치 — 첫 청크 헤더에서 샘플레이트·채널을 파싱해 모든 SSE 청크에 독립 헤더를 재생성, 청크 단위 독립 디코딩 가능하게

실시간 스트리밍 엔드포인트 신설: 수정 후에도 기존 `/sse` 는 전체 합성이 끝난 뒤 WAV를 분할 전송하는 구조라, 텍스트가 길수록 TTFB가 비례해 늘어났다.

엔진의 gRPC server streaming을 활용한 `/sse-realtime` 을 신설해 **문장 단위 합성 → 즉시 전송** 구조로 전환.

정비: API v2 경로 표준화(v1은 deprecated로 하위 호환 유지), Swagger 문서 정비, 구·신 버전 TTFB를 직접 비교하는 데모 페이지 제공.

성과

- **TTFB 1,943ms → 985ms** — 같은 텍스트를 양쪽 엔드포인트로 완주시킨 비교(58.0초 342청크 / 59.4초 364청크), 데모 페이지 콘솔 로그 그대로.
첫 문장이 짧은 런에서는 334ms까지 측정됐으나 단일 런이라 대표값으로 쓰지 않음
- 원인 분석부터 조치·검증·고객 보고까지 단독 수행
- 이 경험이 이후 Qwen3-TTS 플랫폼을 저지연 스트리밍 중심으로 설계하는 출발점이 됨

데모 페이지

방식 1: 구버전(SSE) vs 신버전(SSE-realtime) 직접 비교

두 방식의 차이는 gRPC 호출 방식입니다.
두 버전을 번갈아 눌러 TTFB(버튼 클릭 → 첫 오디오 재생까지 대기 시간)를 직접 비교해보세요.
텍스트가 길수록 차이가 뚜렷하게 납니다.
⚠ 구버전(/sse)은 /api/v1/2와 /api/v2/ 경로 모두 존재합니다.
두 경로 모두 동일한 방식(전체 합성 후 분할 전송)이며 deprecated입니다.

항목	구버전 /sse	신버전 /sse-realtime
gRPC 방식	Unary (단발 호출)	Server Streaming
합성 방식	전체 합성 완료 → WAV 파일 분할 → SSE 전송	문장 단위 합성 → 완성 즉시 SSE 전송
첫 오디오 수신	전체 합성 완료 후	첫 문장 완료 즉시
상태	deprecated	최신
장점	✓ 짧은 텍스트에서 안정적	✓ TTFB 최소화 (첫 문장 즉시 재생) ✓ 긴 텍스트일수록 유리
단점	✗ 전체 합성 완료 전까지 첫 청크 없음 ✗ 긴 텍스트일수록 대기 시간 증가	— (구버전 대비 없음)

구버전 (deprecated)

POST /api/v2/tts-engine/synthesize/sse
(/api/v1/tts-engine/synthesize/sse 도 동일 방식 — 둘 다 deprecated)
전체 합성이 끝날 때까지 첫 청크가 오지 않습니다

구버전 실행

중지

TTFB: 1943ms 총 청크: 342개

포맷: WAV 샘플레이트: 24000 Hz 길이: 58.03s

완료 (342개 청크)

(오류: 10:55:55) 첫 청크 수신 → TTFB: 1943ms
(오류: 10:55:55) 완료: 총 342개 청크

신버전

POST /api/v2/tts-engine/synthesize/sse-realtime
첫 문장이 완성되는 즉시 재생 시작합니다

신버전 실행

중지

TTFB: 985ms 총 청크: 364개

포맷: WAV 샘플레이트: 24000 Hz 길이: 59.43s

완료 (364개 청크)

(오류: 10:55:54) 첫 청크 수신 → TTFB: 985ms
(오류: 10:55:55) 완료: 총 364개 청크

TTS API 샘플

공통 옵션

텍스트

안녕하세요. 음성 합성 테스트입니다.
에이아이체스의 음성개발팀입니다.
샘플 텍스트를 작성하였습니다.
확인 부탁드립니다.
구버전용입니다.

Voice IDkor_male_02 (남성 2)

오디오 포맷 (SSE 실시간은 wav 고정)wav

Speed (재생 속도: 1.0)Pitch (음성 높낮이: 1.0)Volume (음성 크기: 1.0)

API 서버 주소

고객 안내용으로 제공한 데모 페이지 — 동일 텍스트로 구버전(/sse)과 신버전(/sse-realtime)의 TTFB를 직접 비교 체험할 수 있게 구성 (스크린샷의 예시 런: 1943ms → 985ms, 내부 서버 주소는 가림 처리)

STT 배포 MiCo AI

2025.07 ~ 2025.09 · STT 배포 · 서빙 담당 (팀 협업)

문제 대량 상담 전화를 실시간 처리해야 하는데 기존 STT가 채널 부족·추론 지연으로 병목

해결 Triton 기반으로 전처리·후처리·서비스 계층을 분리하고, 파일 배치와 실시간 스트리밍 두 경로를 모두 지원하도록 정비.
파일 경로에는 외부에서 받은 화자 구간 정보를 붙여 화자별 전사를 생성

2개
추론 모델 (파일 · 스트리밍)

HTTP · gRPC
인터페이스 동시 제공

Whisper-large-v3

Faster Whisper

Triton (Python Backend)

Streaming ASR

gRPC / HTTP

Docker

배경

AIG 상담센터의 대량 상담 전화 실시간 처리 필요.

기존 STT 시스템 병목으로 인한 채널 부족 및 추론 지연 불안정.

구현

AIG STT 구조 개선: Triton Inference Server 기반 음성인식 파이프라인을 전처리·후처리·서비스 계층으로 모듈화해 파일/스트리밍 공통 로직을 통합.

YAML 기반 로그 로테이션·아카이빙 체계와 Docker 구성을 정비하고, 컨테이너 자동 재시작 정책으로 안정성을 보강.

엔진 구성: Triton Python 백엔드로 파일 기반과 실시간 스트리밍 두 추론 모델을 구성하고, HTTP와 gRPC 인터페이스를 모두 제공.

화자별 전사: 요청에 담겨 오는 화자 구간(`diarization_data`)을 Faster Whisper의 `clip_timestamps` 형식으로 변환해 화자마다 따로 전사를 돌리고, 결과를 화자별로 묶어 반환.

어느 화자에도 명확히 귀속되지 않는 구간은 `Ambiguous` 로 분리해 오귀속을 막음.

화자 구간 자체는 외부 입력이며, 별도 과제였던 Pyannote 파인튜닝 모델은 채택되지 않아 여기에 연결되지 않았음

성과

- Triton Inference Server 기반 배포 - 고성능 음성인식 파이프라인
- Docker 컨테이너화로 배포 자동화 및 안정성 강화
- 배치 처리와 스트리밍 처리 모두 지원
- 전처리·후처리·서비스 계층 분리로 코드 재사용성 확보

Kubernetes-based AI Infrastructure 인피닉

2024.03 ~ 2025.04 · 3인 인프라 팀 — 관측성 · 스토리지 담당

문제 AI 서비스 20여 개를 수동 배포해 한 번에 수 시간이 걸리고, 장애가 나도 클러스터 어디서 무엇이 터졌는지 볼 수단이 없었음

해결 3인 인프라 팀으로 사내 첫 K8s 플랫폼 구축.

그중 **관측성 전 계층(지표·경보·로그)**과 **스토리지(NFS PV/PVC)**를 맡아, 20여 개 모델이 도는 클러스터를 들여다보고 모델·데이터 볼륨을 관리

5종

직접 구축한 관측 스택

20개 이상

관측 대상 ML 모델

수시간 → 자동화

배포 소요 (팀 성과)

Kubernetes

Prometheus

Grafana

AlertManager

Elasticsearch

Loki

NFS PV/PVC

Docker

ArgoCD

배경

AI 서비스·API 20여 개를 수동 배포해 한 번에 수 시간이 소요.

자동화·스케일링·자원 효율 개선이 필요했고, 클러스터로 옮긴 뒤에는 무엇이 어디서 터졌는지 볼 수단도 함께 있어야 했음.

팀 구성과 담당

인프라 엔지니어 3인이 온프레미스에 사내 첫 Kubernetes 플랫폼을 구축.

아래가 팀이 만든 전체 구성이고, 그중 **관측성 계층이 본인 담당**.

- 클러스터·네트워크 — MetalLB LoadBalancer, Ingress-nginx 라우팅 **팀**
- 스토리지 — NFS 기반 PV/PVC로 모델·데이터 영속 볼륨 관리 **본인**
- 배포 파이프라인 — GitLab → Jenkins·Argo Workflow → Harbor → ArgoCD GitOps **팀**
- 지표·경보 — Prometheus 수집, Grafana 대시보드, AlertManager 경보 규칙 **본인**
- 로그 — Elasticsearch(ELK)와 Loki를 클러스터에 직접 설치해 수집·조회 환경 구성 **본인**

구현

- Prometheus로 클러스터·노드·워크로드 지표를 수집하고, Grafana 대시보드로 20여 개 모델의 상태를 한 화면에서 보게 구성
- AlertManager 경보 규칙을 세워 장애를 사람이 발견하기 전에 알리도록 전환
- 로그는 관리형 서비스 없이 클러스터에 Elasticsearch·Loki를 직접 설치·운영 — 온프레미스라 외부 로깅 SaaS를 쓸 수 없는 조건
- 스토리지 — 용도별로 나뉘어 있던 서버들(모델 가중치 보관, DB 등)을 클러스터로 통합하는 과정이라 저장소가 한 곳이 아니었음. 각 저장소를 NFS로 노출하고 용도별 PV를 만들어 워크로드가 PVC로 바인딩하도록 구성 — 파드가 재시작돼도 모델 가중치를 다시 받지 않게 됨
- 구축 이후 클러스터 장애 대응과 리소스 관리에 팀의 일원으로 참여

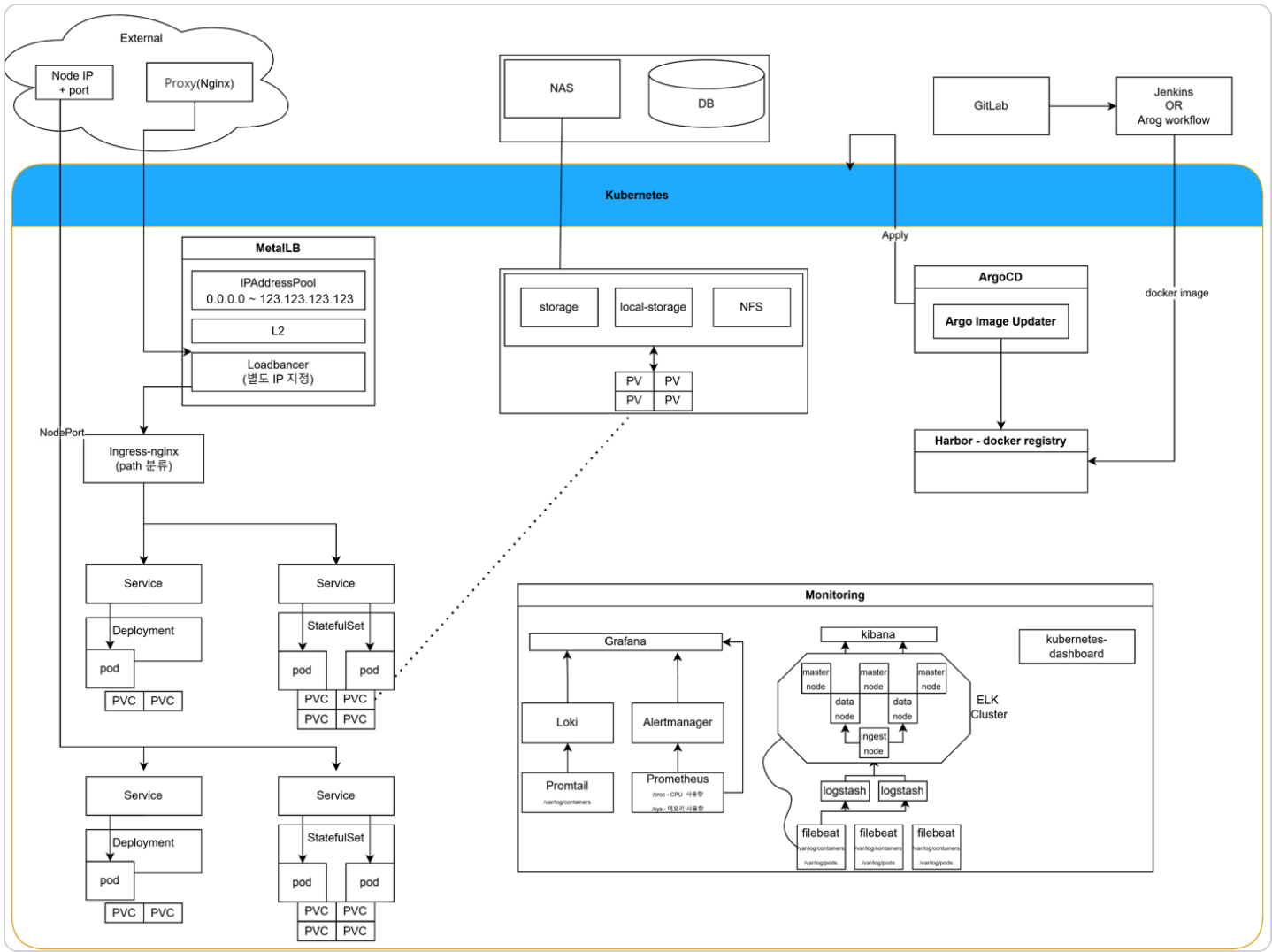
성과

- 3인 인프라 팀으로 **회사 최초 Kubernetes 기반 AI 플랫폼**을 구축하고, 수동 배포를 GitOps 자동화로 전환 **팀 성과**
- 지표와 대시보드, 경보, 로그까지 **관측성 5종을 혼자 구축** (Prometheus, Grafana, AlertManager, Elasticsearch, Loki)
- NFS 기반 PV/PVC로 20여 개 모델의 가중치와 데이터 볼륨을 영속 관리
- 20개 이상 ML 모델이 도는 클러스터를 지표와 로그로 들여다볼 수 있게 만들어, 장애 원인 추적이 가능해짐

돌아보면

- 사내 첫 도입이라 참고할 선례가 없었고, 온프레미스에서 여러 NFS를 용도별로 붙이는 구성은 자료가 드물어 PV·PVC 바인딩 관계를 잡는 데 시행착오가 있었음.
문서와 검색으로 메우는 데 시간을 썼고, 그만큼 남이 읽을 문서를 남기는 습관이 생김
- 관측 스택은 세웠지만 SLO를 정의하지 못했음.
경보를 리소스 임계값에 걸어 뒀서, 사용자가 체감하는 장애와 어긋나는 경우가 있었음.
지금이라면 모델별 지연·에러율로 목표를 먼저 정하고 경보를 거기 걸었을 것

시스템 구조



모델 개발 · 연구 5

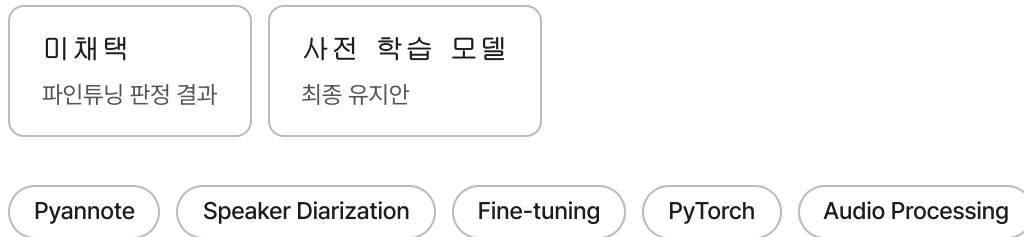
Speaker Diarization with Pyannote MiCo AI

2025.04 ~ 2025.07 · 화자 분리 모델 평가 · 파인튜닝 (미채택)

문제 자사 메신저 제품의 영상통화 회의록 기능에 화자 구분이 필요했으나, STT는 텍스트만 돌려줘 누가 말했는지 알 수 없었음

해결 Pyannote 사전 학습 모델을 평가하고 공개 데이터로 파인튜닝을 시도했으나 DER 개선이 없어 미채택.

STT 결합까지 가지 못하고 종료 — 판단 근거와 회고를 아래 기록



배경

회사가 만드는 메신저·협업 제품에 영상통화 회의록·전사 기능을 넣으려면 발화자 식별이 필요했음.

STT는 음성 인식만 제공해 누가 말했는지 구분할 수단이 없었고, 화자 분리 모델을 검토해 제품에 쓸 수 있는 수준인지 판정하는 것이 과제였음.

구현

모델 평가: Pyannote 사전 학습 모델을 상담 음성에 적용해 DER을 측정하고, 채택 가능한 수준인지 판정하는 것이 1차 목표였음.

파인튜닝 시도: 상담 도메인에 맞추려 공개 화자 분리 데이터를 모아 파라미터 튜닝과 학습을 수행하고 DER로 평가.

성과

- 모델 평가와 파인튜닝 파이프라인(데이터 수집 → 학습 → DER 측정)을 구축해 **채택 여부를 판정할 근거를 만들**.
결론은 미채택이었고, STT 결합까지 진행되지 못한 채 종료
- 파인튜닝은 **사전 학습 모델 대비 DER 개선이 없어 미채택**. 결과를 포장하지 않고 그대로 기록

미채택 기록과 회고

- 확보한 학습 데이터의 양과 라벨이 파인튜닝을 지탱할 수준이 아니었음.
공개 데이터를 모으는 것으로 시작한 시점에 이미 데이터 규모·품질 점검이 선행됐어야 했다고 봄
- 지금이라면 학습부터 하지 않음.
사전 학습 모델의 DER을 먼저 재고, 상담 음성 몇십 건이라도 직접 라벨링해 실패 유형(중첩 발화·짧은 맞장구·채널 특성)을 분류한 뒤, 그 유형이 학습으로 풀리는 문제인지부터 판정했을 것
- 모델을 바꾸는 대신 후처리로 접근하는 선택지도 검증하지 않았음.
상담 통화는 2인 대화라는 사전 지식이 강하게 주어지는데, 화자 수를 2로 고정하고 짧은 구간을 병합하는 규칙만으로도 얼마나 좁혀지는지 재봤어야 했음

NLP Text Summarization System 인피닉

2024.01 ~ 2024.07 · 사내 지식 관리 · 요약 시스템 개발

문제 휴가·부재 후 복귀한 직원이 단체방에 쌓인 업무 대화를 따라잡는 데 시간이 오래 걸림

해결 대화는 BART, 문서는 T5로 나눠 파인튜닝(R3F 정규화)하고 사내 메신저에 탑재

2초 대

업무 대화 요약

약 300명

사내 메신저 전자 탑재

BART

T5

R3F Fine-tuning

Hugging Face

NLP

PyTorch

배경

휴가나 부재 후 복귀한 직원이 단체방에 쌓인 업무 대화를 따라잡는 데 오랜 시간이 걸린다는 문제에서 출발.
사내 메신저에 넣을 한국어 요약 모델 개발이 목표.

구현

모델 설계: 대화 요약은 BART, 문서 요약은 T5로 나눠 개발.
대화 요약 파인튜닝에는 사전 학습 표현을 훼손하지 않는 R3F 정규화 기법을 적용.

데이터 파이프라인: 한국어 대화-요약 쌍 데이터 수집과 전처리, 임베딩 모델 선정까지 학습 파이프라인을 직접 구축.
ROUGE 점수와 사내 인원 휴먼 평가를 품질 지표로 사용.

성과

- 사내 메신저에 요약 기능을 붙여 **전 직원(약 300명)**이 쓰는 서비스에 실제 적용
- 긴 사내 공지 문서를 한 줄 요약으로 압축, 실제 업무 대화는 2초대에 요약
- 카카오톡 대화 요약 기능과 나란히 비교해 유사한 품질의 요약을 확인한 데모 수행
- STT 결과 텍스트를 활용한 자연어 처리 기능 개발과 전처리·평가 환경 구축도 함께 수행

3D Semantic Segmentation Research 인피닉

2022.08 ~ 2022.12 · 센서 퓨전 기반 자율주행 연구 · 모델 개발

문제 2D 세그멘테이션만으로는 자율주행에 필요한 거리·높이 정보를 얻을 수 없음

해결 LiDAR와 카메라를 융합하는 Trans-Unet 설계.

Transformer의 장거리 의존성에 U-Net의 다중 스케일을 결합

2편

논문 (제1저자)

2건

특허 (제1발명자)

Trans-Unet

Sensor Fusion

PyTorch

LiDAR

Computer Vision

Autonomous Driving

배경

자율주행 환경 인식에 3D 거리/높이 정보 필수.

기존 2D 세그멘테이션의 한계로 인한 센서 퓨전 필요.

구현

모델 아키텍처: Trans-Unet (Transformer + U-Net 하이브리드) 설계로 LiDAR 포인트 클라우드 + 카메라 이미지 효과적 퓨전.
Transformer 장거리 의존성 + U-Net 다중 스케일 피쳐 추출 결합.

연구 기여: 논문 2편 제1저자 게재 (한국자동차공학회 2022.11, 2023.04) 및 특허 2건 등록 (제1발명자).

성과

- 논문 2편 게재 (제1저자, 한국자동차공학회 2022.11, 2023.04)
- 특허 2건 등록 (제1발명자, 등록번호: 1025382250000, 1025382310000)

Synthetic Data Generation with Latent Diffusion 인피닉

2023.01 ~ 2023.12 · 생성 모델 기반 국방 데이터 확보 연구

문제 국방 도메인은 보안 제약으로 학습 데이터 수집이 어렵고, GAN은 모드 붕괴로 대안이 못 됨

해결 Latent Diffusion에 조건부 생성과 LoRA를 얹어 도메인 특성을 학습시키고 합성 데이터 생성

우수 논문상

한국국방기술학회 2023

2편

관련 논문

Latent Diffusion

Generative Model

PyTorch

LoRA Fine-tuning

Data Augmentation

배경

국방 도메인 AI 모델 개발의 학습 데이터 부족 (보안 제약으로 수집 어려움).

GAN 모드 붕괴 문제 및 범용 생성 모델의 도메인 불일치.

구현

모델 개발: Latent Diffusion 노이즈 스케줄 + 조건부 생성 + LoRA 파인튜닝.

국방 이미지의 스타일, 배경, 객체 특성 학습으로 현실성 높은 합성 데이터 생성.

검증 및 평가: 합성 데이터를 실제 모델 학습에 적용하여 유효성 검증.

성과

- 우수 논문상 수상: "국방 데이터 확보를 위한 생성모델 Latent Diffusion 실험" (한국국방기술학회 2023.11)
- 합성 데이터 생성 자동화 파이프라인 구축
- 추가 논문: "GAN을 활용한 데이터 생성 연구 동향" (한국항공우주학회 2023)

생성 결과

국방 데이터 확보를 위한 생성 모델 Latent Diffusion을 활용한 이미지 생성.

Image generation using Latent Diffusion, a generation model for securing defense data

이윤선^{1,*}, 김민욱², 최준³

Yoonseon Lee^{1,*}, Yeonmook Kim², Joon Choi³

[주목]

최근 군사 및 방위 분야에서 AI 기술을 접목하여 데이터 확보가 활발해지고 있다. 이러한 AI 기술 적용을 위해서는 대량의 고품질 데이터가 필요하며, 방위 분야에서는 AI 기술의 성능을 높이기 위해 방위 데이터를 활용하는 것이 중요하다. 그러나 방위 데이터의 수집은 매우 어렵고, 방위 데이터의 활용은 방위 분야의 보안과 직결되어 있어, 방위 데이터의 활용을 위해서는 방위 데이터의 수집과 활용을 위한 법적, 제도적 장치가 필요하다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다.

[ABSTRACT]

Recently, attempts to broadly apply AI technology in the military and defense fields have become active. In order to apply such AI technology, a large amount of high-quality data sets are required, and large amounts of high-quality data are very important as they also affect the performance of AI technology. However, in the case of national defense data, there is a difficulty in securing large amounts of data due to security reasons due to the nature of national defense, so there is a problem in that it is not easy to apply the technology. Accordingly, research is currently underway to secure defense data through generative models. In using generative models, studies were actively conducted using GAN-based generative models, but after the announcement of Latent Diffusion, studies applying it showed good performance. In this study, 7,386 tank-related images were collected from the web and used as training data to create images of tanks, one of the important assets of national defense, and new tank images were created using the Latent Diffusion model. Through this, the possibility of generating and supplementing defense data using the generative model was confirmed.

Key Words : Generative model, Latent Diffusion, Diffusion model, Latent Diffusion model, Defense data

1. 서론

최근 AI 기술의 발전으로 인해 국방 분야에서도 AI 기술의 적용이 활발해지고 있다. 특히, 방위 데이터의 수집과 활용은 방위 분야의 보안과 직결되어 있어, 방위 데이터의 활용을 위해서는 방위 데이터의 수집과 활용을 위한 법적, 제도적 장치가 필요하다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다.

국방 데이터 확보를 위한 생성 모델 Latent Diffusion을 활용한 이미지 생성.

Image generation using Latent Diffusion, a generation model for securing defense data

이윤선^{1,*}, 김민욱², 최준³

Yoonseon Lee^{1,*}, Yeonmook Kim², Joon Choi³

[주목]

최근 군사 및 방위 분야에서 AI 기술을 접목하여 데이터 확보가 활발해지고 있다. 이러한 AI 기술 적용을 위해서는 대량의 고품질 데이터가 필요하며, 방위 분야에서는 AI 기술의 성능을 높이기 위해 방위 데이터를 활용하는 것이 중요하다. 그러나 방위 데이터의 수집은 매우 어렵고, 방위 데이터의 활용은 방위 분야의 보안과 직결되어 있어, 방위 데이터의 활용을 위해서는 방위 데이터의 수집과 활용을 위한 법적, 제도적 장치가 필요하다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다.

[ABSTRACT]

Recently, attempts to broadly apply AI technology in the military and defense fields have become active. In order to apply such AI technology, a large amount of high-quality data sets are required, and large amounts of high-quality data are very important as they also affect the performance of AI technology. However, in the case of national defense data, there is a difficulty in securing large amounts of data due to security reasons due to the nature of national defense, so there is a problem in that it is not easy to apply the technology. Accordingly, research is currently underway to secure defense data through generative models. In using generative models, studies were actively conducted using GAN-based generative models, but after the announcement of Latent Diffusion, studies applying it showed good performance. In this study, 7,386 tank-related images were collected from the web and used as training data to create images of tanks, one of the important assets of national defense, and new tank images were created using the Latent Diffusion model. Through this, the possibility of generating and supplementing defense data using the generative model was confirmed.

Key Words : Generative model, Latent Diffusion, Diffusion model, Latent Diffusion model, Defense data

1. 서론

최근 AI 기술의 발전으로 인해 국방 분야에서도 AI 기술의 적용이 활발해지고 있다. 특히, 방위 데이터의 수집과 활용은 방위 분야의 보안과 직결되어 있어, 방위 데이터의 활용을 위해서는 방위 데이터의 수집과 활용을 위한 법적, 제도적 장치가 필요하다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다.

국방 데이터 확보를 위한 생성 모델 Latent Diffusion을 활용한 이미지 생성.

Image generation using Latent Diffusion, a generation model for securing defense data

이윤선^{1,*}, 김민욱², 최준³

Yoonseon Lee^{1,*}, Yeonmook Kim², Joon Choi³

[주목]

최근 군사 및 방위 분야에서 AI 기술을 접목하여 데이터 확보가 활발해지고 있다. 이러한 AI 기술 적용을 위해서는 대량의 고품질 데이터가 필요하며, 방위 분야에서는 AI 기술의 성능을 높이기 위해 방위 데이터를 활용하는 것이 중요하다. 그러나 방위 데이터의 수집은 매우 어렵고, 방위 데이터의 활용은 방위 분야의 보안과 직결되어 있어, 방위 데이터의 활용을 위해서는 방위 데이터의 수집과 활용을 위한 법적, 제도적 장치가 필요하다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다.

[ABSTRACT]

Recently, attempts to broadly apply AI technology in the military and defense fields have become active. In order to apply such AI technology, a large amount of high-quality data sets are required, and large amounts of high-quality data are very important as they also affect the performance of AI technology. However, in the case of national defense data, there is a difficulty in securing large amounts of data due to security reasons due to the nature of national defense, so there is a problem in that it is not easy to apply the technology. Accordingly, research is currently underway to secure defense data through generative models. In using generative models, studies were actively conducted using GAN-based generative models, but after the announcement of Latent Diffusion, studies applying it showed good performance. In this study, 7,386 tank-related images were collected from the web and used as training data to create images of tanks, one of the important assets of national defense, and new tank images were created using the Latent Diffusion model. Through this, the possibility of generating and supplementing defense data using the generative model was confirmed.

Key Words : Generative model, Latent Diffusion, Diffusion model, Latent Diffusion model, Defense data

1. 서론

최근 AI 기술의 발전으로 인해 국방 분야에서도 AI 기술의 적용이 활발해지고 있다. 특히, 방위 데이터의 수집과 활용은 방위 분야의 보안과 직결되어 있어, 방위 데이터의 활용을 위해서는 방위 데이터의 수집과 활용을 위한 법적, 제도적 장치가 필요하다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다. 본 논문에서는 방위 데이터를 수집하고, 방위 데이터를 활용하여 생성된 이미지를 실제 방위 데이터와 비교하여, 방위 데이터의 활용을 위한 법적, 제도적 장치를 마련하는 것을 목표로 한다.



Tank Object Detection Model

인피닉

2023.01 ~ 2023.08 · 전차 탐지 모델 개발

문제 정찰·드론 영상을 사람이 일일이 분석해 시간과 비용이 큼

해결 YOLO 기반 탐지 모델에 조도·각도·배경 증강을 더해 환경이 바뀌어도 탐지가 유지되도록 학습

YOLOv5 / v8
탐지 모델

조도 · 각도 · 배경
강건성 증강 축

- YOLO
- Object Detection
- PyTorch
- Defense Domain

배경

- 정찰/드론 영상에서 전차 자동 탐지 필요.
- 기존 수동 분석의 시간 및 비용 문제.

구현

- 모델 개발:** YOLOv5/v8 기반 전차 탐지 모델.
- 다양한 조도, 각도, 배경 시나리오 강건성 확보를 위한 데이터 증강 및 하이퍼파라미터 튜닝.
- 모델 학습 및 검증:** 국방 도메인 데이터 기반 학습 및 성능 검증.
- 다양한 환경 시나리오에서의 탐지 강건성 확보.

성과

- YOLO 기반 전차 탐지 모델 개발 완료
- 데이터 증강 및 하이퍼파라미터 튜닝으로 탐지 강건성 확보

데이터 · 도구 2

OCR Dataset Annotation Tool 이튼티앤에스

2022.04 ~ 2022.08 · 단독 개발 (Full-Stack) — GUI 도구 설계 · 구현

문제 표 영역 OCR 학습 데이터를 수동 주석으로 만들다 보니 그 작업 자체가 병목

해결 PyQt 주석 도구를 직접 만들어 생성 과정을 자동화하고, 표 탐지·셀 텍스트 추출 모델까지 구현

성과금 수령

프로젝트 성공

PyQt

OCR (Tesseract)

OpenCV

Vision Transformer

Python

배경

PDF/이미지 문서에서 표 영역 추출을 위한 OCR 학습 데이터 필요.
수동 주석 작업의 병목 해결 필요.

구현

시스템 아키텍처: PyQt GUI 기반 Annotation Tool 설계 및 단독 개발.
이미지 로딩, 라벨링, 데이터 저장 기능 구현.

AI 모델 개발: Tesseract 및 ViT 기반 표 영역 탐지, RNN 기반 테이블 셀 내 텍스트 추출 모델 개발.
OCR 학습 데이터 생성 프로세스 자동화.

성과

- Annotation Tool 설계 및 단독 개발 (상용 서비스 투입 전 단계까지 참여)
- OCR 학습 데이터 생성 프로세스 자동화
- OCR 문서 인식 성능 개선
- 프로젝트 성공 성과금 수령

Data Collection & Preprocessing

큐헛지

2020.09 ~ 2021.09 · 인턴 — 데이터 수집 · 전처리 파이프라인 구축

문제 여러 소스에서 모은 금융 데이터를 수동으로 정제·통합하느라 분석까지 시간이 오래 걸림

해결 수집 → 정제 → 검증 → 저장을 자동화한 파이프라인 구축

4단계

수집 · 정제 · 검증 · 저장 자동화

- Python
- Pandas / NumPy
- SQL
- Data Engineering

배경

- 금융 데이터 분석을 위한 고품질 데이터 필수.
- 다중 소스 수집 데이터의 수동 정제 및 통합으로 인한 비효율.

구현

- 데이터 수집 & 정제:** 금융 데이터 크롤링 및 수집.
Pandas/NumPy 기반 결측값 처리, 이상치 제거, 형식 정규화.
- 자동화 파이프라인:** 수집 → 정제 → 검증 → 저장 자동화.
SQL 기반 데이터 관리.

성과

- 금융 데이터 자동 수집 파이프라인 구축
- 데이터 품질 검증 도구 개발
- 데이터 엔지니어링 역량 확보

기술 검증 · 개인 연구

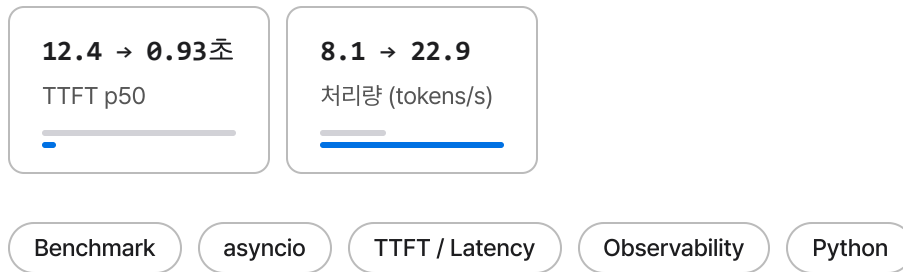
개인 프로젝트 6

llm-bench 개인 프로젝트

2026.07 · LLM 추론 벤치마크 CLI 설계 · 구현

문제 추론 서버 성능을 손으로 재다 보니 병목이 큐잉인지 연산인지 구분되지 않음

해결 동시성 수준별 스윙으로 TTFT와 처리량을 함께 기록해, 처리량이 고정된 채 지연만 느는 큐잉을 분리



배경

TTS 게이트웨이 TTFB를 절반 이하로 줄이며 손으로 하던 측정을 도구화.

OpenAI 호환 엔드포인트(Ollama, vLLM 등)에 동시 부하를 걸어 TTFT·지연 percentile·처리량을 측정하는 CLI.

구현

측정 설계: asyncio + httpx로 동시성 수준별 스윙.

스트리밍 청크 단위로 TTFT(첫 토큰까지 시간)와 tokens/s를 요청별 기록, p50/p95 집계.

예열 요청으로 모델 로드 시간을 측정에서 분리.

의존성은 httpx 하나, 차트는 SVG 직접 생성.

성과

- 로컬 Ollama(CPU) 기준선: 동시성 1에서 TTFT 0.62초, 7.6 tokens/s
- 동시성을 2·4로 올리자 TTFT는 7.9초·12.4초로 악화되는데 처리량은 8.1 tokens/s에 고정.
요청이 병렬로 처리되지 않고 줄을 서고 있다는 신호
- 원인은 Ollama 기본 설정(NUM_PARALLEL=1)의 요청 직렬화.
벤치마크 도구가 병목의 위치를 짚어냄
- NUM_PARALLEL=4로 서버 설정만 바꿔 같은 스윙을 재실행하니 TTFT p50이 12.4초에서 0.93초로, 합산 처리량이 8.1에서 22.9 tokens/s로 스케일.
진단 → 설정 변경 → 재측정으로 루프를 닫음

GitHub 저장소 (<https://github.com/ckc5800/llm-bench>)

LangGraph 기반 Corrective-RAG Agent

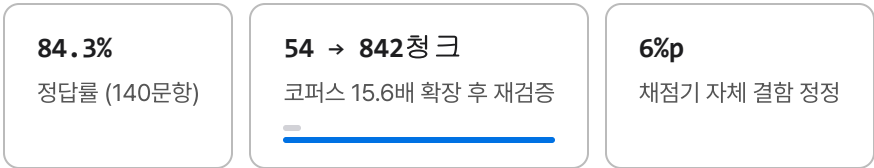
개인 프로젝트

2026.07 · LLM Agent 시스템 설계 · 구현

문제 RAG 정답률이 낮을 때 검색·청킹·생성 중 어디가 원인인지 분리되지 않음

해결 층별 단독 평가 체계를 만들어 병목을 정량 규명.

지표를 쓰기 전에 채점기부터 검증



- LangGraph
- RAG
- Tool Calling
- Hybrid Search (FAISS + BM25)
- Ollama
- FastAPI
- Evaluation

배경

LLM Agent 시스템(RAG, tool orchestration, state 관리)의 E2E 구축 역량 확보 목적.
포트폴리오 기술문서를 지식 베이스로 하는 Q&A Agent를 전부 로컬 환경(Ollama)에서 구현.

구현

- LangGraph StateGraph 기반 Corrective-RAG — retrieve → grade(자가 평가) → 판정 부족 시 rewrite 재검색 → generate
- FAISS(의미) + BM25(키워드) RRF 융합 직접 구현 — 벡터 저장소는 환경변수로 Qdrant 교체 가능, 동일 평가로 비교해 선택 기준 실측
- 에이전트 레이어 3종 — ReAct 도구 호출, Planner-Worker-Synthesizer 멀티홉 분해, MCP 클라이언트
- 인덱싱 전 데이터 검수 CI 연결 — 노션 내부 링크가 코퍼스의 9.4%를 차지하던 오염 발견·제거

성과

- 평가셋을 10 → 51 → 140문항으로 늘려가며 재측정 — 현재 **140문항 84.3%** (포트폴리오 100문항 83% · TTS 40문항 88%).
실패를 검색·청킹·생성 층으로 나눠 병목이 생성 단계임을 수치로 확인하고 컨텍스트 수를 튜닝
- 코퍼스를 **54 → 842청크(15.6배)**로 늘린 뒤 튜닝값을 전수 재검증 — 작은 코퍼스에서 정한 청크 크기·top-N·HyDE 판정이 그대로 유지됨을 확인.
"코퍼스가 작아서 기각했다"던 유보를 실제로 시험한 결과.
검색 지표는 10문항 앵커의 recall@1 90%가 확장 평가에서 착시로 드러나, 결론을 확장본 기준으로 갱신하고 하이브리드 검색 결합 7건을 수정
- 채점기가 동의 표기를 오답으로 세고 있던 것을 발견.
6%p를 정정 — 지표를 쓰기 전에 지표부터 검증한 경우.
기본 모델도 3B → 7B로 전환해 **+8%p(77 → 85%)**를 지연 비용 없이 얻고, 재작성률이 67~76%에서 16%로 떨어지는 것까지 재측정
- 코퍼스에 답이 없는 거부 문항 10개를 넣어 **환각 측정 지표 신설**.
컨텍스트를 넓힐수록 환각이 느는 트레이드오프를 실측

평가 방법론

- 검색/생성 분리 평가 — recall@k(LLM 불필요)와 E2E 정답률을 분리해 실패 원인 귀속
- gold 라벨을 청크 md5·앵커 문자열로 고정 — 재청킹 시 자동 무효화, 전 지표에 무작위 기준선 병기
- 3B 모델의 런 간 편차 ±6%p 실측 — 모든 A/B는 반복 측정, 1~2문항 차이로 결론 내지 않는 원칙
- RAGAS 도입 검토 후 보류 — 로컬 3B 심판의 일치율 45~50%로 정보량이 없음을 실측하고, 재도입 조건(7B 이상·일치율 90%)을 코드에 명시

실험 기록

- Parent-Child 청킹 — 초기엔 유일하게 풀던 집게 질문을 base의 top-N 확장이 함께 해결했고 child recall@1도 열위(78% vs 89%)라 미채택.
판단 근거를 코드 주석에 기록
- 시맨틱 청킹(임베딩 유사도 breakpoint, 직접 구현) — 동일 조건 비교에서 base가 전 지표 우위로 미채택
- 격리 단일 호출의 판단이 전체 파이프라인 반복 측정에서 뒤집힌 사례 확보 — n=1 실험의 함정을 실측으로 문서화
- 이 사이트의 프로젝트 검색이 살아있는 데모 — BM25 레이어를 순수 브라우저 JS로 이식 (다중 단어 AND, 한글 바이그램으로 조사·어미 대응, 관련도순 정렬). 목차의 검색창에서 바로 체험 가능

GitHub 저장소 (코드 및 개선 히스토리) (<https://github.com/ckc5800/rag-agent>)

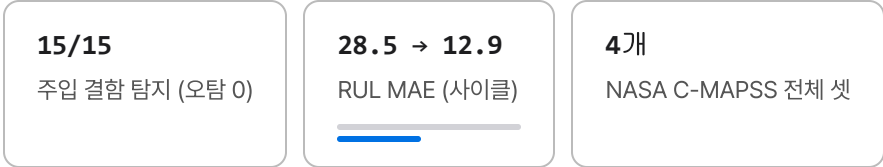
예지보전 Agent (pdm-agent)

개인 프로젝트

2026.07 · 센서 이상 탐지 + LLM 진단 파이프라인 설계 · 구현

문제 LLM에 이상 탐지까지 맡기면 근거 없는 수치를 지어내 정비 판단에 쓸 수 없음

해결 탐지는 결정적 신호처리(z-score·EWMA·추세 회귀)가 맡고, LLM은 탐지 근거 해석만 하도록 역할 분리



- Time Series
- Anomaly Detection
- Signal Processing
- Predictive Maintenance
- LLM

배경

음성 AI에서 실시간 신호를 다뤄온 경험을 산업 설비 시계열로 확장.
회전기계 센서(진동·온도·전류)에서 이상을 탐지하고 LLM이 정비 진단 리포트를 작성하는 예지보전 데모.

구현

설계 원칙 — "탐지는 결정적 알고리즘, LLM은 해석만": 이상 탐지는 z-score(순간 충격)·EWMA 관리한계(과열)·추세 회귀+간이 RUL 추정(베어링 마모)의 신호처리 알고리즘이 담당.
LLM은 탐지 근거를 종합해 원인 추정·권고 조치를 작성하되, 근거 밖 수치 생성을 프롬프트로 차단.

시뮬레이터: 부하 변동을 포함한 회전기계 모델에 결함 3종 주입 — 주입 시점이 ground truth로 남아 탐지기를 정량 평가.

성과

- 5개 시나리오에 주입한 결함 15건을 전부 탐지.
재현율 100%에 오탐 0건이지만, 통제 실험 기준임을 함께 밝힌다
- 추세 탐지기가 "한계치 도달까지 약 29시간" RUL 추정 산출
- LLM 진단이 탐지 근거만으로 베어링 마모/윤활 불량 원인 후보 도출

실데이터 검증 (NASA C-MAPSS)

- 같은 탐지기를 터보팬 엔진 100대의 run-to-failure 벤치마크에 적용 — 합성에서 튜닝한 4σ 한계는 63/100으로 보수적, 3σ 완화 시 **고장 전 경보 94/100, 조기 경보 0건** (관리한계 트레이드오프를 표로 공개)
- RUL은 보정(엔진 1~50)과 평가(51~100)를 분리해 라벨 누수 차단 — 선형 외삽 MAE 28.5사이클을 지수 열화 모델로 **12.9사이클**까지 절반 이하로 개선 (추정 보류 트레이드오프까지 기록)
- **4개 셋 전부로 확장**(통합 CLI) — 운전 조건이 6개인 FD002는 정규화 없이 경보 0/130이었고, 조건별 z-정규화로 104/130 회복.

난이도는 결합 모드 수보다 **운전 조건 수**

- 센서 10개를 하나의 health index로 **융합** — 단일 결합 모드 셋은 대폭 개선(FD001 94→**100/100**, FD002 104→**130/130**), 결합 모드가 2개인 셋은 오히려 악화.
평균 융합의 전제가 깨지는 지점까지 그대로 기록

GitHub 저장소 (<https://github.com/ckc5800/pdm-agent>)

한국어 멀티모달 상품 검색 (visual-search) 개인 프로젝트

2026.08 · 커머스 비주얼 검색 E2E 단독 구축

문제 한국어 문장형 질의는 색·성별 같은 세부 조건에서 임베딩 검색만으로 정확도가 떨어짐
해결 실패를 gold 크기 구간별로 분해해 원인을 먼저 규명한 뒤, 질의에서 속성을 뽑아 메타 필터로 결합

90 → 98%

hit@10 (한국어 50문항)

6종

다국어 전략 실측 비교

44K

상품 인덱스

- CLIP
- Multimodal Retrieval
- Hybrid Search
- MT / LLM Translation
- RRF Ensemble
- FastAPI
- Evaluation

배경

rag-agent에서 구축한 검색 평가 방법론(gold 라벨, 실패 원인 분해, 미채택 실험 기록)을 멀티모달로 확장.
패션 상품 44,072개(공개 데이터셋)를 한국어 문장형 질의("체크무늬 네이비 남성 셔츠")로 검색하는 시스템을 CLIP 임베딩 인덱스부터 평가셋·대화형 데모까지 구축.

구현

- 이미지+텍스트 복합 질의 — "이 상품이랑 비슷한데 검정색으로": 스타일은 임베딩 가중 합성, 속성은 메타 필터가 담당
- 대화형 정교화 — 후속 발화를 조건 병합, 부정어(말고, 빼고) 처리, 대화 경로 UI. 서버는 무상태
- FastAPI + 바닐라 JS 데모 — 검색 응답 약 80ms, 질의 임베딩 LRU 캐시(반복 질의 165 → 0ms), 단계별 타이밍 노출
- 44K 규모라 정확 탐색(numpy 내적) 채택 — ANN 도입 시점을 규모 기준으로 명시

GitHub 저장소 (데모 GIF·평가 리포트·번역문 전수 기록) (<https://github.com/ckc5800/visual-search>)

성과

- 한국어 50문항 × 44K 기준 **hit@10 90% → 98%**.
임베딩 단독의 세부 조건 약점을 구간 분해로 먼저 규명(gold≤50 구간 precision@10 0.10)한 뒤, 질의에서 색·성별·용도를 뽑아 메타 필터로 결합해 해소(0.53)
- 같은 이미지 인덱스에서 텍스트 경로만 바꿔 **다국어 전략 6종을 실측 비교**.
m-clip 48% < 소형 MT 86% < **LLM(3B)+커머스 용어집 92%**. 번역기는 크기순이 아님(NLLB 600M 76% < 8배 작은 opus 86%)
- **대형 멀티링구얼(jina-v2 865M)과는 상보 관계**. 지표는 jina-v2가 앞서지만 임베딩 비용이 ko-clip의 약 130배.
실패 문항도 정확히 갈림. 외래어는 멀티링구얼이, 콩글리시는 한국어 특화가 강세
- gold가 넓은 질의는 hit이 자동으로 나온다. 이 **인플레이션**을 구간별 분해·precision@10 병기로 해소.
결과마다 재현 메타(모델 id·시각·top-10 id) 기록

수요 예측 × 시계열 파운데이션 모델 검증 (demand-forecast) 개인 프로젝트

2026.08 · M5 수요 예측 백테스트·모델 비교 E2E 단독 구축

문제 "제로샷 시계열 파운데이션 모델이 튜닝된 베이스라인을 이기는가"에 대한 실측 근거 부재 — 간헐 수요에선 지표 함정까지 겹침
해결 M5 3만 시리즈 롤링 백테스트로 베이스라인·LightGBM·Chronos를 6라운드 비교 — 판정이 집계 레벨과 지표 선택에 따라 갈림을 실측

역전
바닥(이동평균 1위) ↔ 집계(Chronos 1위)

0.675
전사 RMSSE — 제로샷 전 레벨 1위

학습 0초
CPU 28초에 1,500회 예측

Time Series Forecasting

Chronos (Foundation Model)

LightGBM

Backtesting

Evaluation

배경

시계열 파운데이션 모델(Chronos)의 제로샷 성능 주장을 커머스 수요 데이터에서 직접 검증.

월마트 일별 판매 30,490 시리즈(M5)에 h=28 롤링 오리진 백테스트를 구축하고, 오염 검증(M5가 Chronos 학습 코퍼스가 아닌 공식 제로샷 평가셋임을 확인)부터 선행.

구현

- 롤링 오리진 백테스트 하네스 — RMSSE(M5 공식)·MASE·WQL, 가변 길이 시리즈·시리즈별 시작 요일 대응, 스케일 미정치는 NaN+제외 보고
- 전 시리즈 공용 LightGBM(랙·롤링·요일·카테고리·달력 공변량, tweedie, 재귀 다단 예측) / Chronos-bolt 제로샷 / 시리즈 특성 기반 모델 라우팅
- 예측 곡선 시각화 — 집계 레벨의 주간 패턴 추종 vs 바닥 레벨 간헐 수요를 한 장으로 (재현 스크립트 포함)

GitHub 저장소 (라운드별 실측 기록·검증 리포트) (<https://github.com/ckc5800/demand-forecast>)

성과

- **계층 레벨에서 판정 역전 실측.**
판매 0이 55%인 바닥 레벨은 28일 이동평균이 RMSSE 1위(0.741), 집계 레벨(0 비율 ~0%)은 제로샷 Chronos가 전 레벨 1위 (store 0.717·전사 0.675) — 파운데이션 모델의 가치는 "신호가 있는 곳"에서 크다
- **지표·집계 방식이 결론을 바꾸는 지점 문서화.**
"전부 0 예측"이 naive를 이기는 간헐 수요 함정, MASE/RMSSE 승자 분화, 판매량 가중 시 이동평균-Chronos 동률(0.718 vs 0.720)
- **반론 봉쇄** — LightGBM 스왑 6구성 전수 기록·달력 공변량·집계 레벨별 재설계까지 측정.
학습 모델이 학습 0초 제로샷을 평균에서 못 넘는 구도 확인(시리즈별 승수 4/10 한계 병기)
- **강건성 검증** — 겹치지 않는 대체 샘플·6폴드 확장에서 순위 재현, 컨텍스트 길이 민감도 $\pm 0.5\%$.
코드 감사로 잠재 버그 4건 수정, 회귀 테스트 10개

Portfolio MCP Server 개인 프로젝트

2026.07 · MCP 서버 설계 · 구현

문제 포트폴리오가 문서로만 있어 AI 도구가 경력·프로젝트를 직접 조회할 수 없음

해결 MCP 표준 서버로 read-only 도구 4종을 제공하고, 추론은 클라이언트 LLM에 위임해 서버는 결정적으로 유지

2개

의존성 · clone 후 30초 내 동작

18%

BM25 토큰 감소

MCP

FastMCP

BM25

Pydantic

Python

배경

포트폴리오를 AI 도구 생태계에 연결 — Claude Desktop/Claude Code 등 MCP 클라이언트가 경력·프로젝트·논문 정보를 도구로 조회할 수 있도록 표준 프로토콜 서버 구현.

구현

도구 설계: read-only 도구 4개 — 구조화 조회(프로필/프로젝트/논문·특허) + BM25 문서 검색.

검색 실패 시 다음 행동을 안내하는 actionable 오류 응답 설계.

경량 아키텍처: 의존성 2개(MCP SDK + rank_bm25)만으로 구성 — 임베딩 서버·외부 API·GPU 불필요, clone 후 30초 내 동작. 추론은 클라이언트 LLM에 위임하고 서버는 결정적으로 유지.

- FastMCP 기반 stdio 서버, Pydantic 입력 검증, read-only 어노테이션
- 확정 사실(profile.json)과 서술형 문서 검색을 이원화한 구조
- Claude Code에 사용자 스코프로 등록해 실제 에이전트 세션에서 도구 호출을 검증 — 세션을 재생한 SVG 데모를 README에 수록
- RAG Agent와 동일 지식 베이스 공유 — RAG·MCP 두 노출 방식 비교 경험

성과

- 노션 export의 퍼센트 인코딩 경로가 무의미 토큰으로 쪼개져 길이 정규화 페널티를 유발하던 문제를 원문 정제로 해결 (토큰 7,653 → 6,310개, 18% 감소)
- 응답이 오는지만 보던 스모크 테스트를 내용 단언(프로젝트 수·특허 건수)으로 전환, 과정에서 구 사명 미인식 버그 발견·별칭 매칭 추가
- rag-agent의 에이전트가 이 서버를 클라이언트로 사용.
제공자와 소비자 양쪽을 모두 구현

GitHub 저장소 (<https://github.com/ckc5800/portfolio-mcp>)

대회 · 대외 팀 프로젝트 2

AI Face De-ID (얼굴 정보 자동 비식별화) 팀 프로젝트

2021 · 4인 팀 프로젝트 (데이터 어드벤처 팀)

문제 모자이크 비식별화는 정보를 지워버려 Re-ID·행동 인식 같은 후속 태스크에 쓸 수 없음

해결 얼굴과 랜드마크를 검출한 뒤 생성 모델로 가짜 얼굴을 합성해 대체.

형태는 남기고 신원만 제거

4인 팀

데이터 어드벤처 2021

생성 대체

모자이크 아닌 비식별화

Computer Vision

Face Detection

Generative Model

Privacy / De-ID

배경

공공 데이터 비식별화에 쓰이는 모자이크 처리는 원본 데이터의 정보 손실을 일으켜, Re-ID·행동 인식 등 얼굴 데이터가 필요한 후속 태스크에 활용하기 어려움.

이를 해결하기 위해 **AI로 생성한 가짜 얼굴로 원본 얼굴을 대체**하는 비식별화 기법을 제안.

구현

거리 영상에서 보행자 얼굴 영역 검출 및 랜드마크 검출 → 생성 모델 기반 가짜 얼굴 합성으로 정보 손실을 최소화하면서 개인정보를 보호하는 라벨링 파이프라인.

1. 프로젝트 제안 개요

- 과기정통부의 데이터 담 구축을 통해 사회 안전 등을 목적으로 비식별화 된 공공 데이터를 공개하고 있음
- 하지만 Re-ID, 비정상 행동 인식, 군중 인식, 성별 분류와 같이 비식별 이전의 데이터를 요구하는 Task는 활용성과 중요성에 비해 비식별화 된 공공 데이터를 활용한 개발이 매우 어려움
- AI Face De-ID는 가장 중요한 개인정보인 얼굴 정보를 AI를 통해 생성한 가짜 얼굴로 자동 비식별화 하여 원본 데이터의 정보 손실을 줄이면서 동시에 개인정보를 빠르고 쉽게 보호하는 새로운 라벨링 기법임



[모자이크로 인해 원본 데이터의 손실이 발생한 경우]



[너무 많은 얼굴이 있는 경우 모자이크로 인한 손실이 더 커짐]

5. 결과

- Face Detection Output
 - 얼굴 영역 검출 및 랜드마크 검출



심전도 부정맥 진단 AI 대회 참가

2021 · Heart Disease AI Datathon · 부정맥 진단 트랙 참가 — 모델 · 전처리 개발

문제 심전도 원신호에 기저선 변동·근전도 노이즈가 섞여 그대로 학습하면 판별이 어려움

해결 침식·팽창 연산과 spline 보간으로 기저선을 제거한 뒤, lead I 신호를 1D-CNN으로 이진 분류

과기정통부 · NIA

주최 데이터톤

lead I 5,000차원

1D-CNN 입력

1D-CNN

ECG / Bio-signal

Signal Processing

Time Series

PyTorch

배경

과기정통부·NIA 주최, 연세대·서울대병원 주관 의료 AI 데이터톤.

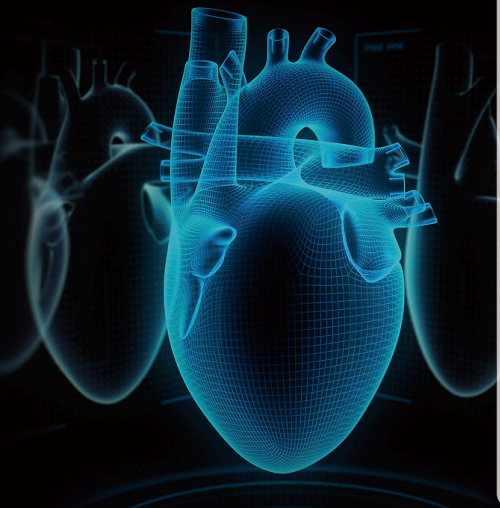
심전도(ECG) 데이터셋으로 부정맥 여부를 진단하는 AI 모델 개발 트랙에 참가.

구현

신호 전처리: 기저선 변동(Baseline Wandering) 제거 — 침식·팽창 연산 및 spline 보간, 주파수 노이즈 제거, 근전도 노이즈는 moving average로 평활화.

모델: 8개 lead 중 심전도 관찰에 유리한 lead 1의 5,000차원 신호를 입력으로, 1-D Convolution Layer(filter 32, kernel 5)를 계층적으로 연결한 1D-CNN으로 주기적 feature를 추출해 정상/부정맥 이진 분류.

HEART DISEASE AI DATATHON 2021



**인공지능 학습용 심장질환 심초음파 및
심전도 데이터셋을 활용한
심초음파/심전도 AI 모델 DATATHON**

대회주제

주제 1 A2C, A4C VIEW에서의 좌심실 분할 AI 모델 개발
주제 2 심전도 데이터셋을 활용한 부정맥 진단 AI 모델 개발
• 각 주제 1.2종씩 1하여 참가하되, 다수의 기술 또는 제품을 보유 중인 경우 중복 참여 가능

대회일정

참가 접수 2021.10.27(수) - 11.25(목)
AI 모델 제출 2021.11.26(금) - 12.07(화)
수상자 발표 2021.12.15(수)

참가자격

의료 인공지능 개발에 관심있는 누구나 (만 15세 이상 내국인, 개인 혹은 5인 이하의 팀 구성)

시상내용

총 상금 2,000만원 / 총 8팀

대상 500만원 (각 주제별 1팀)
최우수상 300만원 (각 주제별 1팀)
우수상 100만원 (각 주제별 2팀)

참가신청

온라인 접수 www.hdaidatathon.com (문의: 홈페이지 내 게시판)

주최 과학기술정보통신부 NIA 한국지능정보사회진흥원 주관 연세대학교 산학협력단 **SNUH** 서울대학교병원 다자발병원 한국판뉴딜

본 대회는 과학기술정보통신부 지원으로 인공지능 학습용 데이터 구축사업 예산을 편성받아 NIA 사업 일환으로 진행됩니다.

논문 · 특허

논문 7편 전편 제1저자

국방 데이터 확보를 위한 생성모델 Latent Diffusion 실험

한국국방기술학회 · 2023.08

우수 논문상 수상 (2023.11)

학습 데이터 부족 문제를 해결하기 위한 Latent Diffusion 기반 데이터 생성 방법론 제시.

국방 도메인의 실제 데이터 확보 문제 해결.

논문 보기 (https://www.kidet.or.kr/index.php?hCode=BOARD&page=view&idx=1219&bo_idx=1)

GAN을 활용한 데이터 생성 연구 동향

한국항공우주학회 · 2023.06

생성 모델의 발전 동향을 분석하고, 데이터 부족 분야에서의 활용 방안 제시.

특히 항공우주 도메인의 실제 적용 사례 제시.

논문 보기 (<https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE11622970>)

센서 퓨전 기반의 Trans-Unet을 활용한 2차원 시맨틱 세그멘테이션의 3차원적 해석

한국자동차공학회 · 2023.04

2D 세그멘테이션 결과를 센서 퓨전을 통해 3D 공간으로 해석하는 방법론 제시.

자율주행 환경 인식의 정확도 향상에 기여.

논문 보기 (<https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE11232100>)

자율 주행 도메인의 3차원 시맨틱 세그멘테이션을 위한 센서 퓨전 기반의 Trans-Unet

한국자동차공학회 · 2022.11

자율주행 자동차의 LiDAR와 카메라 센서를 결합한 센서 퓨전 기반 Trans-Unet 모델을 제안.

3D 환경 인식에서 기존 2D 세그멘테이션 대비 우수한 성능 달성.

논문 보기 (<https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE11220500>)

비정형, 정형 데이터의 이미지 학습을 활용한 시장예측

스마트미디어학회 · 2021.03

정형·비정형 데이터를 이미지로 변환해 딥러닝 기반 시장 예측 모델 개발.

금융 데이터 분석에 새로운 접근법 제시.

논문 보기 (<https://www.kci.go.kr/kciportal/ci/sereArticleSearch/ciSereArtiView.kci?sereArticleSearchBean.artid=ART002735413>)

[PDF 내려받기](#)

Trend of Malware Detection Using Deep Learning

ACM International Conference · 2018.04

딥러닝을 활용한 악성코드 탐지 기법의 발전 동향 분석.

신경망 기반 보안 위협 탐지 방법론 제시.

논문 보기 (<https://dl.acm.org/doi/abs/10.1145/3206129.3239430>)

[PDF 내려받기](#)

Deep Learning을 활용한 악성코드 탐지 방법 동향 분석

한국정보기술학회 · 2018.06

악성코드 탐지의 주요 기법과 최신 동향을 종합적으로 분석.

딥러닝 기반 사이버 보안 솔루션의 효과성 입증.

논문 보기 (<https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE07467643>)

[PDF 내려받기](#)

특허 2건 전건 제1발명자

센서 퓨전을 활용한 3차원 시맨틱 세그멘테이션 방법 및 그 컴퓨터 프로그램

등록번호 10-2538225 · 제1발명자

센서 퓨전을 활용한 3D 시맨틱 세그멘테이션 방법과 그 컴퓨터 프로그램에 관한 발명.

자율주행 환경에서 정확한 공간 인식을 가능하게 함.

특허 보기 (<https://doi.org/10.8080/1020230030194>)

시맨틱 세그멘테이션의 3차원 해석 방법 및 그 컴퓨터 프로그램

등록번호 10-2538231 · 제1발명자

2D 시맨틱 세그멘테이션 결과를 3D 공간으로 해석하는 방법과 그 컴퓨터 프로그램에 관한 발명.

다중 센서 정보를 통합한 3D 환경 인식 기술.

특허 보기 (<https://doi.org/10.8080/1020230030193>)

수상

우수 논문상

한국국방기술학회 추계학술대회 · 2023.11

수상 논문 — "국방 데이터 확보를 위한 생성모델 Latent Diffusion 실험"

학습 데이터 부족 문제를 생성 모델 기반 데이터 확보 방법론으로 해결한 점을 인정받음.